

Notificações por WhatsApp para os responsáveis

Radar — ONG de reforço escolar · documento de arquitetura · julho de 2026

Recomendação: usar a [WhatsApp Cloud API da Meta direto](#) (sem intermediário/BSP), acionada por uma [Edge Function do Supabase](#), com uma tabela de [fila de envio \(outbox\)](#) no Postgres e `pg_cron` disparando os lotes.

Não vale a pena subir uma API Node.js separada *hoje*: ela resolve o mesmo problema, custa mais um servidor, mais um deploy e mais um lugar pra guardar segredo — e o Supabase já é o backend-alvo do Radar. A opção Node está documentada na seção 6, com o critério objetivo de quando migrar.

1. O que a API do WhatsApp exige (inegociável)

Antes de escolher a arquitetura, três regras da Meta definem o desenho. Elas não são opcionais e não têm como contornar:

1. **O token nunca pode ir para o navegador.** A chamada à API do WhatsApp usa um *access token* permanente que dá poder de enviar mensagem em nome da ONG. O Radar é uma SPA com `output: "export"` — qualquer coisa no bundle é pública. Logo, **obrigatoriamente** existe um pedaço de código rodando no servidor. Isso é o que torna a pergunta "Supabase ou Node?" inevitável.
2. **Mensagem iniciada pela ONG precisa de *template* aprovado.** Você não pode escrever texto livre para um número que não falou com você nas últimas 24h. Precisa cadastrar um modelo ("Olá {{1}}, o(a) aluno(a) {{2}} faltou à aula de hoje...") e esperar a Meta aprovar — normalmente minutos, às vezes horas.
3. **Existe uma janela de 24h.** Se o responsável responder a mensagem, abre uma "janela de atendimento" de 24 horas em que você pode mandar texto livre — e de graça. Fora dessa janela, só *template*, e *template* de utilidade é cobrado.

2. Qual provedor: Cloud API, BSP ou biblioteca não-oficial

Caminho	O que é	Veredito para o Radar
Cloud API (Meta)	API oficial hospedada pela própria Meta. Você fala com <code>graph.facebook.com</code> . Sem servidor de WhatsApp pra manter.	✓ Escolher esta. Sem taxa de plataforma, sem mensalidade — você paga só a mensagem para a Meta. É a opção mais barata e a mais estável.
BSP (Twilio, Zenvia, Take Blip, 360dialog, Infobip)	Revendedor oficial. Põe uma API própria por cima da Meta e cobra <i>markup</i> por mensagem ou mensalidade.	Só se a ONG quiser um painel de atendimento humano pronto (chat multi-atendente). Para envio automático, é pagar a mais pelo mesmo serviço.
Não-oficial (Baileys, whatsapp-web.js, Evolution API)	Bibliotecas que simulam o WhatsApp Web logado num celular. Muito comuns no Brasil por serem "de graça".	✗ Não usar. Viola os Termos da Meta, o número pode ser banido sem aviso, e a ONG estaria trafegando dado pessoal de criança por um canal não sancionado. Risco jurídico (LGPD) e operacional que não se paga pelo que economiza.

Por que o Evolution API / Baileys é especialmente ruim aqui

Se o número for banido, a ONG perde o canal de comunicação com *todas* as famílias de uma vez, no meio do semestre, sem recurso. E como o dado é PII de menor de idade, um vazamento por conta de um gateway não-oficial é exatamente o tipo de coisa que a LGPD trata com severidade (art. 14). O custo real da Cloud API é de centavos por mensagem — não vale o risco.

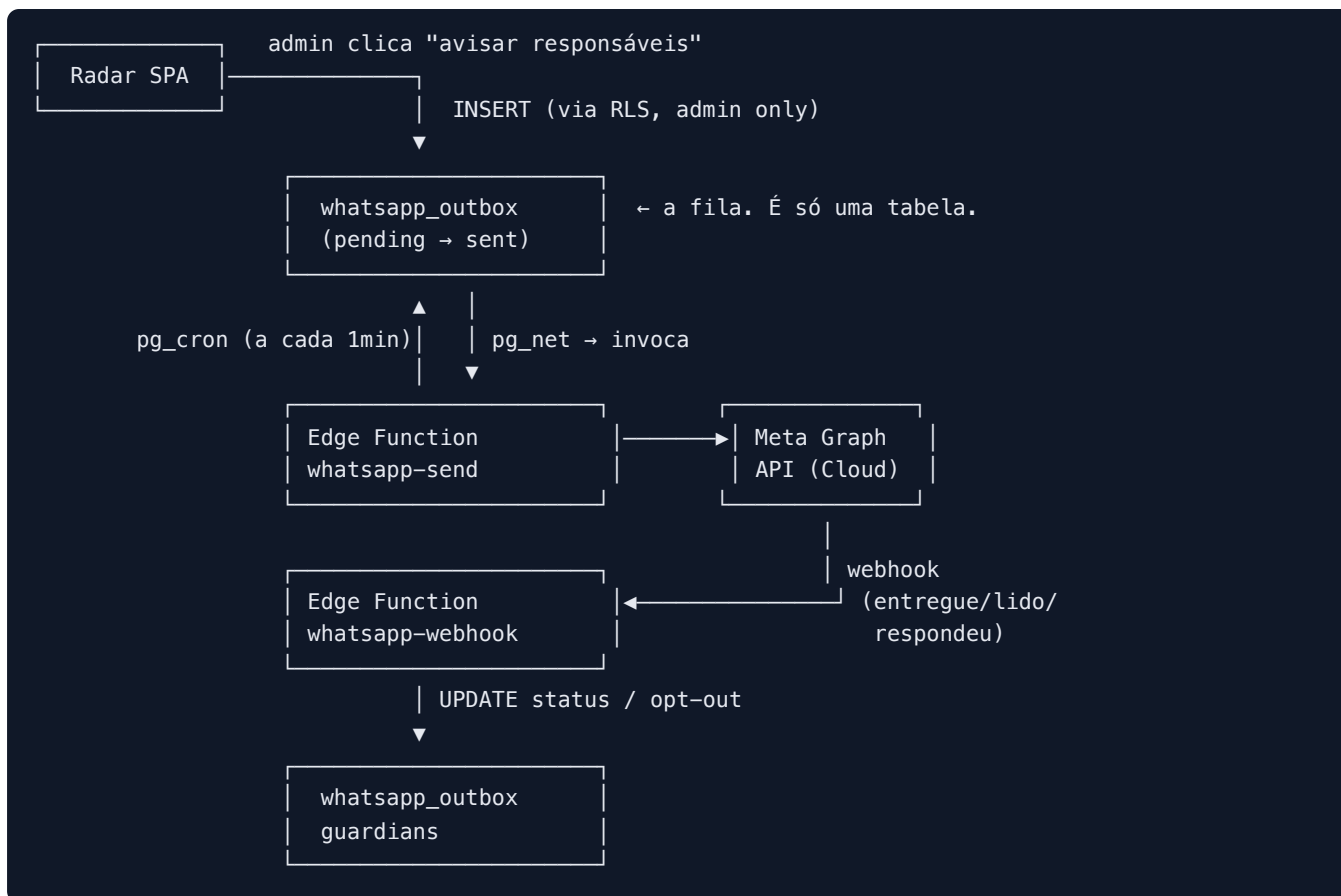
Pré-requisitos de conta (leva ~1 a 3 dias, faça isso primeiro)

1. Conta no **Meta Business Manager** com a ONG (CNPJ).
2. **Verificação de negócio** da Meta (envio de documentos do CNPJ). É o passo mais lento — comece por ele.
3. Criar a **WhatsApp Business Account (WABA)** e registrar um número de telefone. *Atenção:* o número precisa estar limpo — não pode estar em uso num WhatsApp comum ou Business App; ele é migrado para a API e deixa de funcionar no celular.
4. Gerar um **token permanente** via System User (não use o token temporário de 24h da tela de teste).
5. Cadastrar os **templates** e esperar aprovação.

Para desenvolver antes de tudo isso ficar pronto: a Meta dá um **número de teste gratuito** que envia para até 5 destinatários cadastrados. Dá para construir a feature inteira em cima dele.

3. A arquitetura recomendada (Supabase)

O desenho é deliberadamente pequeno: **uma tabela, uma função de envio, uma função de webhook, um cron**. Nada de fila dedicada, worker ou broker.



Por que a tabela outbox existe (e por que não é over-engineering)

A tentação é chamar a API da Meta direto, de forma síncrona, quando o admin clica no botão. Não funciona bem por três motivos concretos:

- **Lote.** "Avisar os responsáveis dos 12 faltantes de hoje" são 12 chamadas HTTP. Se a 7ª falhar, o que aconteceu com as outras? Sem registro, ninguém sabe.
- **Retry.** A Meta devolve 429 e erros transitórios. Sem uma linha com `attempts`, você não tem onde tentar de novo.
- **Auditoria.** É comunicação sobre criança, para responsável legal. Você *precisa* conseguir responder "o que exatamente foi enviado, para qual número, quando, e o WhatsApp confirmou a entrega?". Isso é a tabela.

Uma tabela + um cron é a versão mais barata dessas três coisas. Um BullMQ/Redis aqui seria a versão cara da mesma coisa.

4. Modelo de dados

```
-- Consentimento vive no responsável, não no aluno.
alter table guardians
  add column phone_e164      text,                -- '+5511998887777'
  add column whatsapp_wa_id  text,                -- devolvido pela Meta, ver §8
  add column whatsapp_opt_in boolean not null default false,
  add column opted_out_at   timestamptz;

create type whatsapp_status as enum ('pending','sent','delivered','read','failed');

create table whatsapp_outbox (
  id                uuid primary key default gen_random_uuid(),
  guardian_id       uuid not null references guardians(id),
  student_id        uuid not null references students(id),
  template_name     text not null,                -- 'falta_do_dia'
  params            jsonb not null,               -- {"1":"Maria","2":"João","3":"11/07"}
  status            whatsapp_status not null default 'pending',
  attempts          smallint not null default 0,
  provider_msg_id   text,                        -- wamid.HBg... (id da Meta)
  error             text,
  scheduled_for     timestamptz not null default now(),
  sent_at           timestamptz,
  created_at        timestamptz not null default now(),

  -- Trava de duplicidade: um aviso do mesmo tipo, pro mesmo aluno, no mesmo dia.
  -- É o que impede o pai receber 3 vezes que o filho faltou porque alguém
  -- clicou no botão 3 vezes.
  unique (student_id, template_name, (scheduled_for::date))
);

create index on whatsapp_outbox (status, scheduled_for)
  where status = 'pending';
```

O `unique` é a peça mais importante desse schema

Idempotência no banco, não no código. Se o admin clicar duas vezes, o segundo `INSERT` falha no Postgres — não chega nem perto da Meta. Isso é infinitamente mais confiável que um `if (jáEnviou)` em JavaScript.

RLS

```
alter table whatsapp_outbox enable row level security;

-- Só admin/coordenador enfileira. Professor não dispara mensagem pra família.
create policy "admin enfileira" on whatsapp_outbox for insert to authenticated
with check (
    exists (select 1 from perfis
            where id = auth.uid() and papel in ('admin','coordinator'))
);

-- Ninguém do app atualiza status: só a Edge Function (service_role, bypassa RLS).
create policy "admin lê" on whatsapp_outbox for select to authenticated
using (
    exists (select 1 from perfis
            where id = auth.uid() and papel in ('admin','coordinator'))
);
```

5. As duas Edge Functions

whatsapp-send — puxa da fila e envia

```
// supabase/functions/whatsapp-send/index.ts
import { createClient } from "jsr:@supabase/supabase-js@2";

const GRAPH = `https://graph.facebook.com/v23.0/${Deno.env.get("WA_PHONE_ID")}/messages`;
const TOKEN = Deno.env.get("WA_TOKEN")!;

const db = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!, // fica só aqui, no servidor
);

Deno.serve(async () => {
  const { data: fila } = await db
    .from("whatsapp_outbox")
    .select("id, template_name, params, guardians(phone_e164, whatsapp_opt_in)")
    .eq("status", "pending")
    .lte("scheduled_for", new Date().toISOString())
    .lt("attempts", 3)
    .limit(50); // lote pequeno; o cron volta em 1min

  for (const msg of fila ?? []) {
    if (!msg.guardians.whatsapp_opt_in) {
      await db.from("whatsapp_outbox")
        .update({ status: "failed", error: "sem opt-in" }).eq("id", msg.id);
      continue;
    }

    const res = await fetch(GRAPH, {
      method: "POST",
      headers: { Authorization: `Bearer ${TOKEN}`, "Content-Type": "application/json" },
      body: JSON.stringify({
        messaging_product: "whatsapp",
        to: msg.guardians.phone_e164,
        type: "template",
        template: {
          name: msg.template_name,
          language: { code: "pt_BR" },
          components: [{
            type: "body",
            parameters: Object.values(msg.params).map((t) => ({ type: "text", text: t })),
          }],
        },
      }),
    });

    const body = await res.json();

    await db.from("whatsapp_outbox").update(
      res.ok
        ? { status: "sent", provider_msg_id: body.messages[0].id, sent_at: new Date() }
        : { attempts: msg.attempts + 1, error: body.error?.message ?? "erro" },
    ).eq("id", msg.id);
  }
});
```

```
return new Response("ok");
});
```

whatsapp-webhook — recebe confirmações e opt-out

Esta função é **pública** (a Meta chama de fora). Por isso a assinatura HMAC não é opcional — sem ela, qualquer um marca mensagem como entregue ou descadastra um responsável.

```
// supabase/functions/whatsapp-webhook/index.ts (config.toml: verify_jwt = false)

Deno.serve(async (req) => {
  const url = new URL(req.url);

  // Handshake inicial da Meta ao cadastrar o webhook.
  if (req.method === "GET") {
    return url.searchParams.get("hub.verify_token") === Deno.env.get("WA_VERIFY_TOKEN")
      ? new Response(url.searchParams.get("hub.challenge"))
      : new Response("no", { status: 403 });
  }

  const raw = await req.text();
  if (!(await assinaturaValida(raw, req.headers.get("x-hub-signature-256")))) {
    return new Response("assinatura inválida", { status: 401 });
  }

  const value = JSON.parse(raw).entry?.[0]?.changes?.[0]?.value;

  for (const st of value?.statuses ?? []) {
    await db.from("whatsapp_outbox")
      .update({ status: st.status }) // delivered | read | failed
      .eq("provider_msg_id", st.id);
  }

  // Responsável respondeu "PARAR" → opt-out imediato. Exigência da LGPD e da Meta.
  for (const m of value?.messages ?? []) {
    const texto = m.text?.body?.trim().toUpperCase();
    if (["PARAR", "SAIR", "STOP"].includes(texto)) {
      await db.from("guardians")
        .update({ whatsapp_opt_in: false, opted_out_at: new Date() })
        .eq("whatsapp_wa_id", m.from);
    }
  }

  return new Response("ok"); // sempre 200: a Meta reenvia em erro
});

async function assinaturaValida(corpo: string, header: string | null) {
  if (!header) return false;
  const key = await crypto.subtle.importKey(
    "raw", new TextEncoder().encode(Deno.env.get("WA_APP_SECRET")),
    { name: "HMAC", hash: "SHA-256" }, false, ["sign"],
  );
  const mac = await crypto.subtle.sign("HMAC", key, new TextEncoder().encode(corpo));
  const esperado = "sha256=" + [...new Uint8Array(mac)]
    .map((b) => b.toString(16).padStart(2, "0")).join("");
  return esperado === header;
}
```

O cron

```
select cron.schedule('whatsapp-send', '* * * * *', $$
  select net.http_post(
    url      := 'https://<projeto>.supabase.co/functions/v1/whatsapp-send',
    headers := '{"Authorization": "Bearer <service_role_key>"}'::jsonb
  );
$$);
```

Um minuto de latência é irrelevante para "seu filho faltou hoje". Se um dia precisar ser instantâneo, troca o cron por um trigger `AFTER INSERT` com `pg_net` — mesma função, sem reescrever nada.

6. A alternativa: API Node.js + Postgres próprio

É perfeitamente viável e nada exótico: um Fastify/Hono no Railway, Fly.io ou Render, um Postgres gerenciado (Neon, Supabase, RDS), `pg-boss` para a fila (que usa o próprio Postgres — não precisa de Redis).

O ponto é que ela não resolve nada que o Supabase não resolva — e adiciona: mais um deploy, mais um lugar para vaziar segredo, mais um serviço para monitorar, e o custo de um servidor sempre ligado. Sem contar que o Radar teria dois backends (Supabase para os dados, Node para o WhatsApp) ou uma migração inteira.

Critério	Supabase (Edge Functions)	Node.js + Postgres
Infra a manter	Nenhuma além do Supabase	Servidor + deploy + monitoramento
Custo mensal	R\$ 0 no free tier (500k invocações)	~US\$ 5–20 (servidor + banco)
Onde mora o segredo	Supabase Vault / function secrets	Env do servidor (mais um lugar)
Fila / retry	Tabela + <code>pg_cron</code>	<code>pg-boss</code> (mais robusto)
Auth / RLS	Já é a do Radar	Reimplementar autorização na API
Cold start	~100–300 ms (irrelevante aqui)	Zero
Tarefa longa (>150 s)	Não cabe	Sem limite
Chat ao vivo (atendimento)	Ruim — precisa de conexão persistente	Natural

Critério objetivo para migrar para Node

Migre **quando e se** uma destas for verdade: (a) a ONG quiser um **chat de atendimento ao vivo** pelo WhatsApp, com atendente humano respondendo — isso pede conexão persistente e não cabe bem em serverless; (b) o volume passar de ~5.000 mensagens/dia, quando lote e rate limit ficam um problema de engenharia de verdade; (c) o Radar sair do Supabase por outro motivo. **Enquanto for "avisar falta e mandar boletim", Edge Function resolve e sobra.**

7. Custos reais

Com a Cloud API direto, você paga *só a Meta*, por mensagem. Os avisos do Radar (falta, resumo de frequência, aviso de reunião) se enquadram todos na categoria **Utilidade** — que é a mais barata.

Categoria	Preço (Brasil, USD)	Uso no Radar
Utilidade	~US\$ 0,0068 / msg (≈ R\$ 0,04)	Falta do dia, resumo semanal, alerta de frequência baixa
Serviço (resposta do responsável)	Grátis	Quando o pai responde e você replica dentro de 24h
Marketing	~US\$ 0,0625 / msg	Não usar — 9x mais caro e sujeito a bloqueio
Autenticação	~US\$ 0,0315 / msg	Não se aplica

Simulação para a ONG: 150 alunos, ~10% de falta por dia letivo, 20 dias letivos/mês → **300 mensagens/mês** ≈ **US\$ 2,04** (cerca de **R\$ 11/mês**). Somando um resumo mensal para todos os 150 responsáveis, ainda fica abaixo de **R\$ 30/mês**. Há desconto por volume acima de 1.000 mensagens.

Truque legítimo de custo

Qualquer mensagem enviada dentro de uma janela de atendimento aberta (o responsável respondeu nas últimas 24h) é **grátis** — inclusive templates de utilidade. Vale a pena os templates terminarem com um convite a responder ("Responda esta mensagem se quiser falar com a coordenação") — melhora o relacionamento e derruba o custo das mensagens seguintes.

8. Armadilhas do Brasil (as que custam um dia de debug)

O nono dígito

Números de celular brasileiros registrados no WhatsApp antes de ~2012 têm o `wa_id` **sem o 9 na frente** (5511988887777 vira 551188887777). Você **envia** com o 9 e a Meta resolve — mas o webhook **volta** com o número sem o 9.

Consequência prática: se você casar o webhook com o responsável comparando string de telefone, o opt-out de metade dos pais simplesmente não vai funcionar, silenciosamente. Por isso o schema tem `whatsapp_wa_id` separado de `phone_e164` : **guarde o `wa_id` que a Meta devolveu no primeiro contato e sempre case por ele**, nunca pelo telefone digitado.

- **E.164 sempre.** Guarde `+5511988887777` , não `(11) 98888-7777` . Normalize no `INSERT` , não na hora de enviar.
- **Idioma do template é `pt_BR`** , com underline. `pt-BR` falha.
- **Template rejeitado** por parecer marketing: evite "confira", "aproveite", emoji promocional. Escreva seco e informativo — é o que a categoria Utilidade espera.
- **Rate limit.** Um número novo começa com limite baixo (1.000 destinatários únicos/dia), que sobe conforme a qualidade. Para a escala da ONG isso nunca vai apertar, mas é o motivo do `limit(50)` por

lote.

- **Qualidade do número.** Se muitos pais bloquearem/denunciarem, a Meta rebaixa a qualidade e no limite bloqueia o número. Opt-in de verdade é a defesa — não importe uma planilha de telefones e comece a disparar.

9. LGPD e opt-in

O dado aqui é sensível por natureza: **informação sobre criança, enviada ao responsável legal**. A LGPD (art. 14) exige consentimento específico e destacado de pelo menos um dos pais ou responsável. Na prática:

- **Opt-in explícito e registrado.** Na matrícula, um campo "Autorizo receber avisos sobre meu filho(a) pelo WhatsApp" — e o Radar grava *quando* e *quem* autorizou. Nunca assuma consentimento por ter o telefone no cadastro.
- **Opt-out em um passo.** Responder "PARAR" descadastra na hora (já está no webhook acima). É exigência da Meta e da LGPD.
- **Minimização.** O template manda o primeiro nome do aluno e a data. Não manda endereço, nota, diagnóstico, nem nada que não seja necessário para o aviso.
- **Nunca logue o conteúdo.** O `console.log` da Edge Function vai para o painel do Supabase. Logue `outbox.id` e status — nunca nome de aluno ou telefone completo (regra que o Radar já tem em `CLAUDE.md §7`).

10. Templates a cadastrar

Nome	Categoria	Corpo
<code>falta_do_dia</code>	Utilidade	Olá, {{1}}. Informamos que {{2}} não compareceu à aula de {{3}} no dia {{4}}. Qualquer dúvida, responda esta mensagem. — Equipe [ONG]
<code>alerta_frequencia</code>	Utilidade	Olá, {{1}}. A frequência de {{2}} está em {{3}}% neste mês. Gostaríamos de conversar sobre como podemos ajudar. Responda esta mensagem para falar com a coordenação. — Equipe [ONG]
<code>resumo_mensal</code>	Utilidade	Olá, {{1}}. Resumo de {{2}} em {{3}}: frequência {{4}}%, média {{5}}. Responda esta mensagem se quiser conversar. — Equipe [ONG]

11. Ordem de implementação

1. **Hoje:** abrir o Meta Business, mandar os documentos da ONG para verificação. É o gargalo — o resto se faz em paralelo.
2. Migração do Radar para Supabase (já é o backend-alvo; a feature de WhatsApp depende dela).
3. Campos de `guardians` (telefone, opt-in) + tela de consentimento na ficha do aluno. **Funciona sem nenhuma API do WhatsApp** — é só cadastro.
4. Tabela `whatsapp_outbox` + RLS.

5. Edge Function `whatsapp-send` contra o **número de teste** da Meta (5 destinatários, grátis). Aqui a feature já funciona ponta a ponta.
6. Edge Function `whatsapp-webhook` + HMAC + opt-out.
7. `pg_cron`, botão "avisar responsáveis" na tela de chamada, e troca do número de teste pelo número real da ONG.

Os passos 3 e 4 já entregam valor sozinhos (a ONG passa a ter telefone e consentimento organizados) e não dependem da verificação da Meta terminar.

Radar — documento de arquitetura · julho de 2026

Preços conferidos em julho de 2026 na documentação oficial da Meta; a tabela muda periodicamente — confirme em developers.facebook.com/documentation/business-messaging/whatsapp/pricing antes de fechar orçamento.